Clinic API

Rapport Technique du Microservice

Gestion des Patients d'une Clinique Medicale

Projet :	clinic-api
Version :	0.0.1
Framework :	Spring Boot 3.2.3
Langage :	Java 17
Base de donnees :	H2 (in-memory)
Universite :	Hassan II de Casablanca (FSBM)

Projet realise par :

EL Mehdaoui Chaimaa *(Responsable)*
Maroua Asraoui
Jihane Benzouarhia
Nouhaila Anissi
Dahir Assia
Ibtissam Baissar
Ilhame Naim
Islame Rostom
Walid Ait Sidi Ahmed

Encadre par :

Prof. Chemlal Yman

Table des matières

1 Presentation du Projet **3**
1.1 Contexte et Objectifs . . . 3
1.2 Technologies Utilisees . . . 3
1.3 Structure du Projet . . . 4
1.3.1 Arborescence des Fichiers . . . 4
1.3.2 Architecture en Couches . . . 5

2 Modele de Donnees **6**
2.1 Entite Patient . . . 6
2.1.1 Code Source Complet – Patient.java . . . 6
2.1.2 Point cle : `@JsonIgnore` et confidentialite . . . 8
2.1.3 Schema SQL – schema.sql . . . 8
2.1.4 Donnees Initiales – data.sql . . . 9

3 Architecture et Couches Applicatives **10**
3.1 Point d'Entree – ClinicApiApplication.java . . . 10
3.2 Repository – PatientRepository . . . 10
3.3 Service – PatientService . . . 11
3.4 Controleur – PatientController . . . 13
3.5 Configuration Swagger – SwaggerConfig.java . . . 15

4 Gestion des Exceptions **17**
4.1 Architecture de Gestion des Erreurs . . . 17
4.2 PatientNotFoundException . . . 17
4.3 Structure ErrorResponse . . . 18
4.4 GlobalExceptionHandler . . . 18
4.5 Tableau des Codes HTTP et Cas Declencheurs . . . 19

5 Documentation API – Endpoints **21**
5.1 Vue d'ensemble . . . 21
5.2 Details des Endpoints . . . 21
5.2.1 GET /api/patients — Liste complete . . . 21
5.2.2 GET /api/patients/{id} — Recuperation par ID . . . 22
5.2.3 POST /api/patients — Creation . . . 22
5.2.4 PUT /api/patients/{id} — Modification . . . 22
5.2.5 DELETE /api/patients/{id} — Suppression . . . 23
5.2.6 GET /api/patients/search/secu ?secu={15 chiffres} . . . 23
5.2.7 GET /api/patients/search/nom ?nom={partiel} . . . 23

6 Tests d'Integration **24**

6.1 Tests JUnit – ClinicApiApplicationTests.java 24
6.1.1 Description des 7 Tests 26
6.2 Collection Postman – Tests Manuels 26
6.2.1 Cas Nominaux (7 requetes) 26
6.2.2 Cas d'Erreur (6 requetes) 27

7 Configuration et Deploiement **28**
7.1 Fichier pom.xml – Dependances Maven 28
7.2 Configuration application.properties 29
7.3 URLs d'acces 30
7.4 Lancement du Projet 30

8 Securite et Bonnes Pratiques **32**
8.1 Protection des Donnees Confidentielles 32
8.2 Injection par Constructeur 32
8.3 Validation des Entrees 33
8.4 Gestion Centralisee des Erreurs 33

9 Conclusion **34**
9.1 Bilan 34
9.2 Pistes d'Evolution 34

Chapitre 1

Presentation du Projet

1.1 Contexte et Objectifs

Le projet **Clinic API** est un microservice RESTful developpe dans le cadre du module de developpement backend a l'Universite Hassan II de Casablanca (Faculte des Sciences Ben M'Sik — FSBM). Il simule un systeme de gestion des dossiers patients d'une clinique medicale, expose sous forme d'une API HTTP conforme aux standards REST.

Ce projet pedagogique couvre l'ensemble de la chaine de developpement d'un backend Spring Boot : de la modelisation des donnees a la documentation automatique, en passant par la validation des entrees, la gestion centralisee des erreurs et les tests d'integration.

Objectifs principaux :

- Gestion CRUD complete des patients (Create, Read, Update, Delete)
- Recherche par numero de securite sociale (15 chiffres) et par nom partiel
- Protection des donnees confidentielles (champ `diagnosticConfidentiel` masque dans les reponses JSON via `@JsonIgnore`)
- Documentation automatique et interactive via Swagger / OpenAPI 3
- Validation des entrees utilisateur cote serveur (Jakarta Validation)
- Gestion centralisee et normalisee des erreurs HTTP
- Tests d'integration Spring Boot couvrant les cas nominaux et les cas d'erreur

1.2 Technologies Utilisees

Composant	Technologie	Role
Framework	Spring Boot 3.2.3	Socle du microservice
Langage	Java 17	Langage d'implementation
Persistance	Spring Data JPA	Acces et abstraction des donnees

Composant	Technologie	Role
Base de donnees	H2 (in-memory)	Stockage en memoire (dev/test)
Validation	Spring Validation	Contraintes Jakarta sur les entrees
Documentation	SpringDoc OpenAPI 2.3.0	Swagger UI auto-genere
Serialisation	Jackson (`@JsonIgnore`)	Masquage des donnees sensibles
Build	Maven (mvnw)	Gestion des dependances et build
Tests	JUnit 5 + Spring Boot Test	Tests d'integration contexte complet

1.3 Structure du Projet

1.3.1 Arborescence des Fichiers

Voici l'arborescence complete du projet telle qu'elle est organisee dans le ZIP fourni :

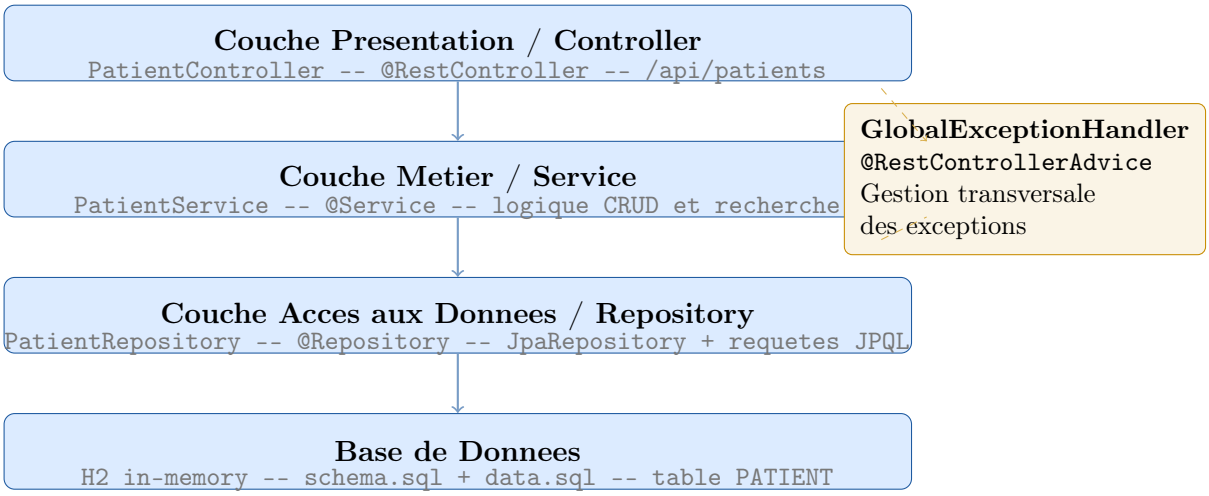
Listing 1.1 – Arborescence du projet clinic-api

```
clinic-api/
|-- pom.xml                          # Descripteur Maven (
   dependances, build)
|-- mvnw / mvnw.cmd                  # Wrapper Maven (lancement
   sans install)
|-- HELP.md                          # Documentation de demarrage
    Spring Boot
|-- src/
|   |-- main/
|   |   |-- java/com/clinic/clinic_api/
|   |   |   |-- ClinicApiApplication.java      # Point d'entree
   @SpringBootApplication
|   |   |   |-- SwaggerConfig.java             # Configuration
   OpenAPI / Swagger
|   |   |   |-- model/
|   |   |   |   `-- Patient.java              # Entite JPA
|   |   |   |-- dao/
|   |   |   |   `-- PatientRepository.java    # Repository Spring
    Data JPA
|   |   |   |-- service/
|   |   |   |   `-- PatientService.java       # Couche metier (
   CRUD + recherche)
|   |   |   `-- web/
|   |   |       |-- controller/
|   |   |       |   `-- PatientController.java # Controleur REST
   (7 endpoints)
|   |   |       `-- exceptions/
```

```
|   |   |           |-- PatientNotFoundException.java  #
   Exception metier 404
|   |   |           |-- GlobalExceptionHandler.java    #
   @RestControllerAdvice
|   |   |           `-- ErrorResponse.java             #
   Structure erreur JSON
|   |   `-- resources/
|   |       |-- application.properties         # Configuration
   Spring Boot
|   |       |-- schema.sql                     # DDL : creation
   table PATIENT
|   |       `-- data.sql                       # DML : 5 patients
    initiaux
|   `-- test/
|       `-- java/com/clinic/clinic_api/
|           `-- ClinicApiApplicationTests.java # 7 tests d'
   integration
`-- clinic-api-postman-collection.json         # Collection
   Postman (13 tests)
```

1.3.2 Architecture en Couches

Le projet suit l'architecture en couches classique de Spring Boot, garantissant une separation claire des responsabilites :



Le flux d'une requete HTTP est le suivant : le **Controller** recoit la requete, la delegue au **Service** qui applique la logique metier, lequel s'appuie sur le **Repository** pour acceder a la base de donnees H2. En cas d'erreur, le `GlobalExceptionHandler` intercepte l'exception et retourne une reponse JSON normalisee.

Chapitre 2

Modele de Donnees

2.1 Entite Patient

L'entite centrale du projet est `Patient`, mappee sur la table SQL `PATIENT`. Elle regroupe les informations administratives du patient ainsi qu'un champ medical confidentiel.

Champ	Type Java	Contraintes	Description
`id`	`Long`	PK, AUTO	Identifiant unique auto-incremente
`nom`	`String`	NotBlank, 2-100	Nom de famille du patient
`prenom`	`String`	NotBlank, 2-100	Prenom du patient
`numeroSecu`	`String`	\d{15}, UNIQUE	Numero de securite sociale (15 chiffres)
`diagnosticConfidentiel`	`String`	@JsonIgnore	Diagnostic medical (masque dans l'API)

2.1.1 Code Source Complet – Patient.java

Listing 2.1 – Entite JPA Patient avec annotations de validation et de documentation

```java
package com.clinic.clinic_api.model;

import com.fasterxml.jackson.annotation.JsonIgnore;
import io.swagger.v3.oas.annotations.media.Schema;
import jakarta.persistence.*;
import jakarta.validation.constraints.*;

@Entity
@Table(name = "PATIENT")
@Schema(description = "Entite representant un patient de la clinique")
public class Patient {

```

```java
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Schema(description = "Identifiant unique du patient",
        example = "1")
    private Long id;

    @NotBlank(message = "Le nom est obligatoire")
    @Size(min = 2, max = 100,
          message = "Le nom doit contenir entre 2 et 100
              caracteres")
    @Schema(description = "Nom du patient", example = "Dupont")
    private String nom;

    @NotBlank(message = "Le prenom est obligatoire")
    @Size(min = 2, max = 100,
          message = "Le prenom doit contenir entre 2 et 100
              caracteres")
    @Schema(description = "Prenom du patient", example = "Jean")
    private String prenom;

    @NotNull(message = "Le numero de securite sociale est
        obligatoire")
    @Pattern(regexp = "\\d{15}",
             message = "Le numero de secu doit contenir
                 exactement 15 chiffres")
    @Schema(description = "Numero de securite sociale (15
        chiffres)",
            example = "175126012345699")
    private String numeroSecu;

    @JsonIgnore     // Ce champ est EXCLU de toutes les reponses
        JSON de l'API
    @Schema(description = "Diagnostic confidentiel (masque dans
        l'API)",
            hidden = true)
    private String diagnosticConfidentiel;

    // ===== Constructeurs =====
    public Patient() {}

    public Patient(String nom, String prenom,
                   String numeroSecu, String
                       diagnosticConfidentiel) {
        this.nom = nom;
        this.prenom = prenom;
        this.numeroSecu = numeroSecu;
        this.diagnosticConfidentiel = diagnosticConfidentiel;
    }

    // ===== Getters & Setters =====
    public Long getId() { return id; }
```

```java
    public void setId(Long id) { this.id = id; }
    public String getNom() { return nom; }
    public void setNom(String nom) { this.nom = nom; }
    public String getPrenom() { return prenom; }
    public void setPrenom(String prenom) { this.prenom = prenom; }
    public String getNumeroSecu() { return numeroSecu; }
    public void setNumeroSecu(String s) { this.numeroSecu = s; }
    public String getDiagnosticConfidentiel() { return diagnosticConfidentiel; }
    public void setDiagnosticConfidentiel(String d) {
        this.diagnosticConfidentiel = d;
    }

    @Override
    public String toString() {
        return "Patient{id=" + id + ", nom='" + nom + "', prenom='"
            + prenom + "', numeroSecu='" + numeroSecu + "'}";
    }
}
```

2.1.2 Point cle : `@JsonIgnore` et confidentialite

L'annotation `@JsonIgnore` sur le champ `diagnosticConfidentiel` est un element fondamental de securite du projet. Elle garantit que le diagnostic medical n'apparait **jamais** dans aucune reponse JSON de l'API, quelle que soit l'operation (GET, POST, PUT).

> **Comportement important :** lors d'un `POST` ou `PUT`, le client *peut* envoyer le champ `diagnosticConfidentiel` dans le corps de la requete (il sera bien sauvegarde en base), mais la reponse retournee ne le contiendra **jamais**. Ce test est couvert explicitement dans `ClinicApiApplicationTests`.

2.1.3 Schema SQL – schema.sql

Listing 2.2 – schema.sql – DDL de creation de la table PATIENT

```sql
DROP TABLE IF EXISTS PATIENT;

CREATE TABLE PATIENT (
    id                      BIGINT AUTO_INCREMENT PRIMARY KEY,
    nom                     VARCHAR(100) NOT NULL,
    prenom                  VARCHAR(100) NOT NULL,
    numero_secu             VARCHAR(15)  NOT NULL UNIQUE,
    diagnostic_confidentiel VARCHAR(500)
);
```

La convention de nommage SQL utilise les *underscores* (`numero_secu`, `diagnostic_confidentiel`) tandis que Java utilise le *camelCase* (`numeroSecu`, `diagnosticConfidentiel`). Spring Data JPA gere cette conversion automatiquement via la strategie de nommage par defaut (`NamingStrategy`).

2.1.4 Donnees Initiales – data.sql

Listing 2.3 – data.sql – Jeu de donnees initial (5 patients)

```sql
INSERT INTO PATIENT (nom, prenom, numero_secu,
   diagnostic_confidentiel) VALUES
('Dupont',  'Jean',   '175126012345699', 'Diabete de type 2'),
('Martin',  'Marie',  '285034065432100', 'Hypertension arterielle
   '),
('Bernard', 'Pierre', '359128078901200', 'Asthme chronique'),
('Dubois',  'Sophie', '467215034567800', 'Migraine chronique'),
('Thomas',  'Luc',    '578302090123400', 'Allergie aux arachides'
   );
```

Ces donnees sont chargees automatiquement au demarrage grace aux proprietes `spring.sql.init.mode=` et `spring.jpa.hibernate.ddl-auto=none`. Le schema est cree avant les donnees (ordre : `schema.sql` puis `data.sql`).

Chapitre 3

Architecture et Couches Applicatives

3.1 Point d'Entree – ClinicApiApplication.java

Listing 3.1 – ClinicApiApplication.java – Point d'entree Spring Boot

```java
package com.clinic.clinic_api;

import org.springframework.boot.SpringApplication;
import
   org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ClinicApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(ClinicApiApplication.class, args);
    }
}
```

L'annotation `@SpringBootApplication` active trois mecanismes : la configuration automatique (`@EnableAutoConfiguration`), le scan des composants (`@ComponentScan`) et la definition de beans de configuration (`@Configuration`).

3.2 Repository – PatientRepository

Le repository etend `JpaRepository<Patient, Long>` et beneficie ainsi automatiquement des operations CRUD de base (`findAll`, `findById`, `save`, `deleteById`, etc.). Des methodes metier supplementaires sont declarees pour les besoins specifiques du projet.

Listing 3.2 – PatientRepository.java – Interface Spring Data JPA

```java
package com.clinic.clinic_api.dao;

import com.clinic.clinic_api.model.Patient;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
```

```java
import org.springframework.stereotype.Repository;
import java.util.List;
import java.util.Optional;

@Repository
public interface PatientRepository extends JpaRepository<Patient, Long> {

    // Methode derivee : Spring genere la requete SQL automatiquement
    // a partir du nom de la methode (findBy + NomDuChamp)
    Optional<Patient> findByNumeroSecu(String numeroSecu);

    // Requete JPQL personnalisee : recherche insensible a la casse
    // LOWER() des deux cotes garantit la recherche case-insensitive
    @Query("SELECT p FROM Patient p WHERE LOWER(p.nom) " +
           "LIKE LOWER(CONCAT('%', :nom, '%'))")
    List<Patient> rechercherParNom(@Param("nom") String nom);

    // Requete par correspondance exacte nom + prenom (insensible casse)
    @Query("SELECT p FROM Patient p WHERE " +
           "LOWER(p.nom) = LOWER(:nom) AND " +
           "LOWER(p.prenom) = LOWER(:prenom)")
    Optional<Patient> findByNomAndPrenom(
        @Param("nom") String nom,
        @Param("prenom") String prenom);
}
```

Methodes derivees vs JPQL : Spring Data JPA peut generer les requetes SQL automatiquement a partir du nom de la methode (`findByNumeroSecu`). Pour des requetes plus complexes (recherche partielle LIKE, combinaison de champs), l'annotation `@Query` avec du JPQL (Java Persistence Query Language) est utilisee, ce qui reste independant du dialecte SQL cible.

3.3 Service – PatientService

La couche service encapsule toute la logique metier. Elle est le seul consommateur du repository et le seul fournisseur de donnees pour le controleur.

Listing 3.3 – PatientService.java – Couche metier complete

```java
package com.clinic.clinic_api.service;

import com.clinic.clinic_api.dao.PatientRepository;
import com.clinic.clinic_api.model.Patient;
```

```java
import com.clinic.clinic_api.web.exceptions.PatientNotFoundException;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class PatientService {

    // Injection par constructeur (bonne pratique Spring)
    private final PatientRepository repository;

    public PatientService(PatientRepository repository) {
        this.repository = repository;
    }

    // ===== READ =====
    public List<Patient> getAllPatients() {
        return repository.findAll();
    }

    public Patient getPatientById(Long id) {
        return repository.findById(id)
                .orElseThrow(() -> new PatientNotFoundException(id));
    }

    // ===== CREATE =====
    public Patient createPatient(Patient patient) {
        return repository.save(patient);
    }

    // ===== UPDATE =====
    // Mise a jour complete (PUT) : on recupere l'entite existante,
    // on ecrase ses champs, puis on sauvegarde.
    public Patient updatePatient(Long id, Patient updated) {
        Patient existing = getPatientById(id); // 404 si absent
        existing.setNom(updated.getNom());
        existing.setPrenom(updated.getPrenom());
        existing.setNumeroSecu(updated.getNumeroSecu());
        existing.setDiagnosticConfidentiel(
            updated.getDiagnosticConfidentiel());
        return repository.save(existing);
    }

    // ===== DELETE =====
    public void deletePatient(Long id) {
        getPatientById(id); // verifie l'existence, lance 404 si absent
        repository.deleteById(id);
    }
```

```java

    // ===== RECHERCHE METIER =====
    public Patient findByNumeroSecu(String numeroSecu) {
        return repository.findByNumeroSecu(numeroSecu)
                .orElseThrow(() -> new PatientNotFoundException(
                    "numero secu = " + numeroSecu));
    }

    public List<Patient> findByNom(String nom) {
        return repository.rechercherParNom(nom);
    }
}
```

3.4 Controleur – PatientController

Le controleur REST expose tous les endpoints sur le chemin de base `/api/patients`. Chaque methode est documentee avec les annotations Swagger OpenAPI.

Listing 3.4 – PatientController.java – Controleur REST complet

```java
package com.clinic.clinic_api.web.controller;

import com.clinic.clinic_api.model.Patient;
import com.clinic.clinic_api.service.PatientService;
import io.swagger.v3.oas.annotations.*;
import io.swagger.v3.oas.annotations.media.*;
import io.swagger.v3.oas.annotations.responses.*;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/patients")
@Tag(name = "Patients",
     description = "API de gestion des patients de la clinique
        medicale")
public class PatientController {

    private final PatientService service;

    public PatientController(PatientService service) {
        this.service = service;
    }

    @GetMapping
    @Operation(summary = "Obtenir tous les patients",
```

```
                description = "Retourne la liste complete de tous les patients")
    @ApiResponse(responseCode = "200",
                 description = "Liste retournee avec succes")
    public ResponseEntity<List<Patient>> getAll() {
        return ResponseEntity.ok(service.getAllPatients());
    }

    @GetMapping("/{id}")
    @Operation(summary = "Obtenir un patient par ID")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description = "Patient trouve"),
        @ApiResponse(responseCode = "404",
                     description = "Patient non trouve",
                     content = @Content(schema = @Schema(
                         implementation = ErrorResponse.class)))
    })
    public ResponseEntity<Patient> getById(
            @Parameter(description = "ID du patient", example = "1")
            @PathVariable Long id) {
        return ResponseEntity.ok(service.getPatientById(id));
    }

    @PostMapping
    @Operation(summary = "Creer un nouveau patient")
    @ApiResponses({
        @ApiResponse(responseCode = "201",
                     description = "Patient cree avec succes"),
        @ApiResponse(responseCode = "400",
                     description = "Donnees invalides")
    })
    public ResponseEntity<Patient> create(
            @Valid @RequestBody Patient patient) {
        return ResponseEntity.status(HttpStatus.CREATED)
                             .body(service.createPatient(patient));
    }

    @PutMapping("/{id}")
    @Operation(summary = "Modifier un patient existant")
    @ApiResponses({
        @ApiResponse(responseCode = "200",
                     description = "Patient modifie avec succes"),
        @ApiResponse(responseCode = "404",
                     description = "Patient non trouve"),
        @ApiResponse(responseCode = "400",
                     description = "Donnees invalides")
    })
    public ResponseEntity<Patient> update(
```

```
            @PathVariable Long id,
            @Valid @RequestBody Patient patient) {
        return ResponseEntity.ok(service.updatePatient(id,
            patient));
    }

    @DeleteMapping("/{id}")
    @Operation(summary = "Supprimer un patient")
    @ApiResponses({
        @ApiResponse(responseCode = "204",
                     description = "Patient supprime"),
        @ApiResponse(responseCode = "404",
                     description = "Patient non trouve")
    })
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        service.deletePatient(id);
        return ResponseEntity.noContent().build();
    }

    @GetMapping("/search/secu")
    @Operation(summary = "Rechercher par numero de securite
        sociale")
    public ResponseEntity<Patient> searchBySecu(
            @Parameter(description = "Numero de secu (15
                chiffres)",
                       example = "175126012345699")
            @RequestParam String secu) {
        return ResponseEntity.ok(service.findByNumeroSecu(secu));
    }

    @GetMapping("/search/nom")
    @Operation(summary = "Rechercher des patients par nom
        (partiel)")
    @ApiResponse(responseCode = "200",
                 description = "Liste de patients
                    correspondants")
    public ResponseEntity<List<Patient>> searchByNom(
            @Parameter(description = "Nom ou partie du nom",
                       example = "Dup")
            @RequestParam String nom) {
        return ResponseEntity.ok(service.findByNom(nom));
    }
}
```

3.5 Configuration Swagger – SwaggerConfig.java

Listing 3.5 – SwaggerConfig.java – Configuration OpenAPI

```
package com.clinic.clinic_api;
```

```java

import io.swagger.v3.oas.models.OpenAPI;
import io.swagger.v3.oas.models.info.*;
import org.springframework.context.annotation.*;

@Configuration
public class SwaggerConfig {

    @Bean
    public OpenAPI clinicOpenAPI() {
        return new OpenAPI()
            .info(new Info()
                .title("Clinic API - Gestion des Patients")
                .description("Microservice Spring Boot pour la gestion des " +
                 "patients d'une clinique medicale. " +
                 "Projet FSBM - Universite Hassan II de Casablanca.")
                .version("1.0.0")
                .contact(new Contact()
                    .name("Equipe Clinique")
                    .email("clinique@fsbm.ma")));
    }
}
```

Chapitre 4

Gestion des Exceptions

4.1 Architecture de Gestion des Erreurs

Le projet dispose d'un systeme de gestion des erreurs centralise et entierement decouple du code metier. Il repose sur trois composants :

`PatientNotFoundException` Exception metier lancee par le service lorsqu'un patient est introuvable (par ID ou par numero secu). Etend `RuntimeException` (exception non verifiee).

`ErrorResponse` Structure Java serialisee en JSON qui contient toujours les trois champs : `timestamp`, `status` et `message`.

`GlobalExceptionHandler` Intercepteur global annote `@RestControllerAdvice` qui convertit chaque type d'exception en reponse HTTP appropriee.

4.2 PatientNotFoundException

Listing 4.1 – PatientNotFoundException.java – Exception metier 404

```java
package com.clinic.clinic_api.web.exceptions;

public class PatientNotFoundException extends RuntimeException {

    // Constructeur avec ID numerique
    public PatientNotFoundException(Long id) {
        super("Patient non trouve avec l'id : " + id);
    }

    // Constructeur avec message libre (ex: recherche par numero secu)
    public PatientNotFoundException(String message) {
        super("Patient non trouve : " + message);
    }
}
```

4.3 Structure ErrorResponse

Listing 4.2 – ErrorResponse.java – Structure JSON normalisee des erreurs

```java
package com.clinic.clinic_api.web.exceptions;

import io.swagger.v3.oas.annotations.media.Schema;
import java.time.LocalDateTime;

@Schema(description = "Structure de reponse d'erreur")
public class ErrorResponse {

    @Schema(description = "Horodatage de l'erreur")
    private LocalDateTime timestamp;

    @Schema(description = "Code HTTP de l'erreur", example =
       "404")
    private int status;

    @Schema(description = "Message d'erreur",
            example = "Patient non trouve avec l'id : 99")
    private String message;

    public ErrorResponse(int status, String message) {
        this.timestamp = LocalDateTime.now(); // horodatage
           automatique
        this.status    = status;
        this.message   = message;
    }

    public LocalDateTime getTimestamp() { return timestamp; }
    public int getStatus()              { return status;    }
    public String getMessage()          { return message;   }
}
```

Exemple de reponse JSON produite lors d'un `GET /api/patients/9999` :

```json
{
  "timestamp": "2025-03-15T14:32:07.123",
  "status": 404,
  "message": "Patient non trouve avec l'id : 9999"
}
```

4.4 GlobalExceptionHandler

Listing 4.3 – GlobalExceptionHandler.java – Gestionnaire global des exceptions

```java
package com.clinic.clinic_api.web.exceptions;

```

```java
import org.springframework.http.*;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.*;
import org.springframework.web.bind.annotation.*;
import java.util.stream.Collectors;

@RestControllerAdvice
public class GlobalExceptionHandler {

    // Handler 404 : Patient introuvable
    @ExceptionHandler(PatientNotFoundException.class)
    public ResponseEntity<ErrorResponse> handlePatientNotFound(
            PatientNotFoundException ex) {
        return ResponseEntity
                .status(HttpStatus.NOT_FOUND)
                .body(new ErrorResponse(404, ex.getMessage()));
    }

    // Handler 400 : Echec de validation @Valid
    // Collecte tous les champs invalides et les concatene
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(
            MethodArgumentNotValidException ex) {
        String errors = ex.getBindingResult()
                .getFieldErrors()
                .stream()
                .map(e -> e.getField() + ": " +
                    e.getDefaultMessage())
                .collect(Collectors.joining(" | "));
        return ResponseEntity
                .status(HttpStatus.BAD_REQUEST)
                .body(new ErrorResponse(400, errors));
    }

    // Handler 500 : Toute autre exception inattendue
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneral(Exception
        ex) {
        return ResponseEntity
                .status(HttpStatus.INTERNAL_SERVER_ERROR)
                .body(new ErrorResponse(500,
                    "Erreur interne du serveur: " +
                        ex.getMessage()));
    }
}
```

4.5 Tableau des Codes HTTP et Cas Declencheurs

Code	Statut	Cas declencheur
200	OK	Requete GET ou PUT reussie ; objet(s) retourne(s)
201	Created	Creation `POST` reussie ; nouvel objet retourne
204	No Content	Suppression `DELETE` reussie ; corps vide
400	Bad Request	Validation `@Valid` echouee (nom vide, secu invalide...)
404	Not Found	Patient introuvable par ID ou par numero secu
500	Server Error	Exception inattendue non geree specifiquement

Chapitre 5

Documentation API – Endpoints

5.1 Vue d'ensemble

Tous les endpoints sont accessibles sous le prefixe `/api/patients`. L'API respecte les conventions REST (verbes HTTP semantiques, codes de statut normalisés).

Methode	URL	Code succes	Description
GET	/api/patients	200 OK	Liste tous les patients
GET	/api/patients/{id}	200 OK	Recupere un patient par ID
POST	/api/patients	201 Created	Cree un nouveau patient
PUT	/api/patients/{id}	200 OK	Modifie un patient existant
DELETE	/api/patients/{id}	204 No Content	Supprime un patient
GET	/api/patients/search/secu	200 OK	Recherche par n° secu
GET	/api/patients/search/nom	200 OK	Recherche par nom (partiel)

5.2 Details des Endpoints

5.2.1 GET /api/patients — Liste complete

Retourne le tableau JSON de tous les patients enregistres en base. Le champ `diagnosticConfidentiel` est systematiquement absent de chaque objet JSON grace a `@JsonIgnore`.

200 OK – Reponse type pour 5 patients initiaux :

```
[
```

```
  {"id": 1, "nom": "Dupont",  "prenom": "Jean",   "numeroSecu
     ": "175126012345699"},
  {"id": 2, "nom": "Martin",  "prenom": "Marie",  "numeroSecu
     ": "285034065432100"},
  {"id": 3, "nom": "Bernard", "prenom": "Pierre", "numeroSecu
     ": "359128078901200"},
  {"id": 4, "nom": "Dubois",  "prenom": "Sophie", "numeroSecu
     ": "467215034567800"},
  {"id": 5, "nom": "Thomas",  "prenom": "Luc",    "numeroSecu
     ": "578302090123400"}
]
```

5.2.2 GET /api/patients/{id} — Recuperation par ID

Retourne le patient dont l'identifiant correspond au parametre de chemin `id`.

200 OK : `GET /api/patients/1` retourne l'objet Dupont Jean.

404 Not Found : `GET /api/patients/9999` retourne `{"timestamp": "...", "status": 404, "message": "Patient non trouve avec l'id : 9999"}`

5.2.3 POST /api/patients — Creation

Cree un nouveau patient. Le corps de la requete doit etre un JSON valide. Tous les champs soumis a validation (`@Valid`) sont verifies avant persistance.

Corps de requete attendu :

```
{
  "nom": "Nouveau",
  "prenom": "Patient",
  "numeroSecu": "999888777666555",
  "diagnosticConfidentiel": "Ceci est confidentiel"
}
```

201 Created : Patient cree et retourne sans le champ `diagnosticConfidentiel` :

```
{"id": 6, "nom": "Nouveau", "prenom": "Patient", "numeroSecu":
    "999888777666555"}
```

400 Bad Request : Si `nom` est vide, la reponse est : `{"status": 400, "message": "nom: Le nom est obligatoire"}`

5.2.4 PUT /api/patients/{id} — Modification

Mise a jour complete du patient identifie par `id`. Tous les champs sont ecrases. Meme corps JSON que pour le `POST`.

200 OK : Patient modifie retourne avec les nouvelles valeurs.

404 si l'ID n'existe pas — **400** si les donnees sont invalides.

5.2.5 DELETE /api/patients/{id} — Suppression

Supprime definitivement le patient. Verifie d'abord l'existence (lance 404 si absent) puis appelle `repository.deleteById()`.

204 No Content : Suppression reussie, corps de reponse vide.

404 Not Found : Patient inexistant.

5.2.6 GET /api/patients/search/secu ?secu={15 chiffres}

Recherche un patient unique par son numero de securite sociale. Le numero doit correspondre exactement (15 chiffres, correspondance exacte).

200 OK : Exemple — `?secu=175126012345699` retourne Dupont Jean.

404 Not Found : Aucun patient ne correspond a ce numero secu.

5.2.7 GET /api/patients/search/nom ?nom={partiel}

Recherche insensible a la casse sur le nom, avec correspondance partielle (LIKE). Exemple : `?nom=dup` trouve "Dupont".

200 OK : Tableau JSON (eventuellement vide) des patients correspondants.

Chapitre 6

Tests d'Integration

6.1 Tests JUnit – ClinicApiApplicationTests.java

Le projet contient une suite de tests d'integration qui demarrent le contexte Spring Boot complet (`@SpringBootTest`), executent les operations sur la base H2 en memoire et verifient les resultats via JUnit 5 (`assertNotNull`, `assertEquals`, `assertThrows`).

Listing 6.1 – ClinicApiApplicationTests.java – Suite de tests d'integration

```java
package com.clinic.clinic_api;

import com.clinic.clinic_api.dao.PatientRepository;
import com.clinic.clinic_api.model.Patient;
import com.clinic.clinic_api.service.PatientService;
import com.clinic.clinic_api.web.exceptions.PatientNotFoundException;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import java.util.List;
import static org.junit.jupiter.api.Assertions.*;

@SpringBootTest
class ClinicApiApplicationTests {

    @Autowired
    private PatientService patientService;

    @Autowired
    private PatientRepository patientRepository;

    // Test 1 : Le contexte Spring se charge sans erreur
    @Test
    void contextLoads() {
        assertNotNull(patientService);
    }

```

```java
    // Test 2 : getAllPatients() retourne bien les 5 patients du data.sql
    @Test
    void testGetAllPatientsReturnsFivePatients() {
        List<Patient> patients = patientService.getAllPatients();
        assertEquals(5, patients.size(),
            "Il doit y avoir 5 patients en base (data.sql)");
    }

    // Test 3 : Recuperer un patient existant par ID
    @Test
    void testGetPatientByIdFound() {
        Patient p = patientService.getPatientById(1L);
        assertNotNull(p);
        assertEquals("Dupont", p.getNom());
    }

    // Test 4 : Tenter de recuperer un patient inexistant lance bien 404
    @Test
    void testGetPatientByIdNotFound() {
        assertThrows(PatientNotFoundException.class,
            () -> patientService.getPatientById(9999L));
    }

    // Test 5 : Creer puis supprimer un patient (cycle complet)
    @Test
    void testCreateAndDeletePatient() {
        Patient newPatient = new Patient(
            "TestNom", "TestPrenom",
            "123456789012345", "Diagnostic test");
        Patient saved = patientService.createPatient(newPatient);
        assertNotNull(saved.getId());

        patientService.deletePatient(saved.getId());

        // Verification que le patient n'existe plus
        assertThrows(PatientNotFoundException.class,
            () -> patientService.getPatientById(saved.getId()));
    }

    // Test 6 : Recherche par numero de securite sociale
    @Test
    void testFindByNumeroSecu() {
        Patient p = patientService.findByNumeroSecu("175126012345699");
        assertNotNull(p);
        assertEquals("Dupont", p.getNom());
    }

    // Test 7 : @JsonIgnore - le diagnostic existe en base
```

```
    // (accessible via repository) mais sera masque dans l'API
    @Test
    void testDiagnosticConfidentielIsNotNull() {
        Patient p = patientRepository.findById(1L).orElseThrow();
        assertNotNull(p.getDiagnosticConfidentiel(),
            "Le diagnostic doit etre present en base");
        // Note : il sera absent des reponses JSON grace a
           @JsonIgnore
    }
}
```

6.1.1 Description des 7 Tests

#	Methode de test	Comportement verifie	Assertion JUnit
T1	`contextLoads()`	Le contexte Spring Boot se charge correctement	`assertNotNull`
T2	`testGetAllPatientsReturnsFivePatients()`	5 patients depuis `data.sql`	`assertEquals(5,...)`
T3	`testGetPatientByIdFound()`	Patient ID=1 existe et s'appelle Dupont	`assertEquals`
T4	`testGetPatientByIdNotFound()`	ID=9999 lance `PatientNotFoundException`	`assertThrows`
T5	`testCreateAndDeletePatient()`	Creation puis suppression d'un patient	`assertNotNull` + `assertThrows`
T6	`testFindByNumeroSecu()`	Recherche par secu trouve Dupont	`assertEquals`
T7	`testDiagnosticConfidentielIsNotNull()`	Diagnostic present en base (mais masque API)	`assertNotNull`

6.2 Collection Postman – Tests Manuels

La collection Postman (`clinic-api-postman-collection.json`) permet de tester l'API manuellement depuis un client HTTP. Elle est divisee en deux groupes.

6.2.1 Cas Nominaux (7 requetes)

#	Nom	Methode / URL	Resultat Attendu
1	Tous les patients	GET /api/patients	200 + tableau JSON (5 patients)
2	Patient par ID (id=1)	GET /api/patients/1	200 + Dupont Jean
3	Creer un patient	POST /api/patients	201 + objet cree (sans diagnostic)

#	Nom	Methode / URL	Resultat Attendu
4	Modifier patient (id=2)	PUT /api/patients/2	200 + objet modifie
5	Supprimer (id=5)	DELETE /api/patients/5	204 No Content
6	Recherche par secu	GET /search/secu ?secu=175126012345699	200 + Dupont Jean
7	Recherche par nom	GET /search/nom ?nom=dup	200 + [Dupont]

6.2.2 Cas d'Erreur (6 requetes)

#	Scenario	Methode	Resultat Attendu
E1	Patient inexistant	GET /api/patients/9999	404 + JSON {status :404, message :...}
E2	Nom vide (`nom=""`)	POST /api/patients	400 + "nom : Le nom est obligatoire"
E3	NumeroSecu trop court	POST /api/patients	400 + message validation
E4	NumeroSecu avec lettres	POST /api/patients	400 + "doit contenir 15 chiffres"
E5	Supprimer inexistant	DELETE /api/patients/9999	404 + JSON d'erreur
E6	Secu inexistant	GET /search/secu ?secu=000000000000000	404 + JSON d'erreur

Chapitre 7

Configuration et Deploiement

7.1 Fichier pom.xml – Dependances Maven

Listing 7.1 – pom.xml – Descripteur Maven (extrait des dependances)

```
<project>
  <groupId>com.clinic</groupId>
  <artifactId>clinic-api</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <description>Gestion d'une Clinique Medicale</description>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.3</version>
  </parent>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <!-- Serveur HTTP + MVC REST -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Persistance JPA / Hibernate -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>

    <!-- Validation Jakarta (@NotBlank, @Size, @Pattern...) -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
```

```
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>

    <!-- Base de donnees H2 in-memory (scope runtime) -->
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <scope>runtime</scope>
    </dependency>

    <!-- Swagger UI / OpenAPI 3 -->
    <dependency>
      <groupId>org.springdoc</groupId>
      <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId
        >
      <version>2.3.0</version>
    </dependency>

    <!-- Tests d'integration -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

Dependance	Version	Role
spring-boot-starter-web	3.2.3	Serveur Tomcat embarque + Spring MVC
spring-boot-starter-data-jpa	3.2.3	Hibernate + Spring Data Repositories
spring-boot-starter-validation	3.2.3	Validation Jakarta (annotations)
h2 (runtime)	3.2.3	Base de donnees relationnelle in-memory
springdoc-openapi-starter-webmvc-ui	2.3.0	Interface Swagger auto-generee
spring-boot-starter-test	3.2.3	JUnit 5 + Spring Test + Mockito

7.2 Configuration application.properties

Listing 7.2 – src/main/resources/application.properties

```
spring.application.name=clinic-api
```

```
# ===== Base de donnees H2 en memoire =====
spring.datasource.url=jdbc:h2:mem:clinicdb
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect

# Console H2 accessible via navigateur
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# ===== Initialisation SQL =====
# always : execute schema.sql + data.sql a chaque demarrage
spring.sql.init.mode=always
# none   : JPA ne touche pas au schema (gere par schema.sql)
spring.jpa.hibernate.ddl-auto=none

# ===== Swagger / OpenAPI 3 =====
springdoc.api-docs.path=/api-docs
springdoc.swagger-ui.path=/swagger-ui.html
```

7.3 URLs d'acces

Interface	URL
API principale	http://localhost:8080/api/patients
Swagger UI (documentation interactive)	http://localhost:8080/swagger-ui/index.html
Documentation OpenAPI JSON	http://localhost:8080/api-docs
Console H2 (base de donnees)	http://localhost:8080/h2-console

Parametres de connexion H2 Console :

- **JDBC URL :** `jdbc:h2:mem:clinicdb`
- **Username :** `sa`
- **Password :** (laisser vide)

7.4 Lancement du Projet

Listing 7.3 – Commandes de lancement et de build

```
# === Lancement en mode developpement ===

# Avec le wrapper Maven fourni (aucune installation requise)
cd clinic-api
```

```
./mvnw spring-boot:run          # Linux / macOS
mvnw.cmd spring-boot:run        # Windows

# Ou via Maven installe globalement
mvn spring-boot:run

# === Build en fichier JAR executable ===
./mvnw clean package
java -jar target/clinic-api-0.0.1-SNAPSHOT.jar

# === Executer les tests uniquement ===
./mvnw test
```

Port par defaut : L'application demarre sur le port **8080**. Pour changer de port, ajouter `server.port=9090` dans `application.properties`.

Base H2 en memoire : Les donnees sont perdues a chaque redemarrage. `schema.sql` et `data.sql` sont rejoues a chaque lancement pour reinitialiser l'etat.

Chapitre 8

Securite et Bonnes Pratiques

8.1 Protection des Donnees Confidentielles

Un element fondamental de ce projet est la gestion de la confidentialite du diagnostic medical. L'annotation `@JsonIgnore` sur le champ `diagnosticConfidentiel` garantit que cette information n'est **jamais exposee** dans les reponses JSON de l'API, meme si elle est bien stockee et accessible en base de donnees.

> **Principe applique :** Le diagnostic est stocke en base (pour usage interne du systeme) mais systematiquement masque dans toute reponse HTTP, conformement au principe du *moindre privilege*. Cela signifie que meme un `GET /api/patients/1` ne retourne pas le diagnostic de Dupont Jean.

Ce comportement est explicitement verifie par le test `testDiagnosticConfidentielIsNotNull()` qui accede directement au repository (bypass de l'API) pour confirmer que le champ est bien non-nul en base, demonstrant ainsi que `@JsonIgnore` masque la donnee au niveau de la serialisation JSON et non au niveau de la base de donnees.

8.2 Injection par Constructeur

Le projet utilise systematiquement l'injection de dependances par constructeur plutot que par champ `@Autowired`. C'est la bonne pratique recommandee par l'equipe Spring car :

- Elle rend les dependances **obligatoires** et visibles (un constructeur avec des parametres non-optionnels)
- Elle facilite les **tests unitaires** (on peut instancier la classe sans conteneur Spring)
- Elle garantit l'**immutabilite** des champs (`private final`)
- Elle evite les cycles de dependances detectes a la compilation

8.3 Validation des Entrees

Toutes les entrees utilisateur sont validees cote serveur via `@Valid` dans le controleur et les annotations de contraintes Jakarta sur l'entite :

- `@NotBlank` : champs `nom` et `prenom` obligatoires, non vides et non constitues uniquement d'espaces
- `@Size(min=2, max=100)` : longueur minimale et maximale des chaines
- `@Pattern(regexp="\d{15}")` : format strict du numero de securite sociale — exactement 15 chiffres
- `@NotNull` : presence obligatoire du numero de secu (meme si vide)

En cas d'echec, le `GlobalExceptionHandler` collecte *toutes* les erreurs de validation et les retourne en une seule reponse 400 concatenee avec `" | "`.

8.4 Gestion Centralisee des Erreurs

Le `@RestControllerAdvice` centralise la gestion des exceptions et offre plusieurs avantages :

- **DRY (Don't Repeat Yourself) :** La logique de gestion d'erreurs n'est pas dupliquee dans chaque methode de controleur
- **Coherence :** Toutes les erreurs suivent la meme structure JSON (`ErrorResponse`)
- **Maintenabilite :** Ajouter un nouveau type d'exception se fait en un seul endroit
- **Semantique HTTP :** Les codes de statut sont semantiquement corrects (404 pour "non trouve", 400 pour "donnees invalides", etc.)

Chapitre 9

Conclusion

9.1 Bilan

Le microservice **Clinic API** constitue une implementation complete, coherente et bien structuree d'une API RESTful Spring Boot. Il couvre l'integralite des couches applicatives d'un backend moderne :

— **Couche donnees :** Entite JPA `Patient` avec contraintes de validation et schema SQL explicite
— **Couche acces :** Repository Spring Data JPA avec methodes derivees et requetes JPQL personnalisees
— **Couche metier :** Service centralisant toute la logique CRUD et de recherche
— **Couche presentation :** Controleur REST exposant 7 endpoints documentes OpenAPI
— **Couche transversale :** Gestion des erreurs centralisee, confidentialite des donnees, validation des entrees
— **Tests :** 7 tests d'integration Spring Boot + 13 scenarios de tests Postman
— **Documentation :** Interface Swagger UI auto-generee accessible en production de developpement

9.2 Pistes d'Evolution

1. **Securite :** Ajout de Spring Security avec authentification JWT ou OAuth2 pour proteger les endpoints sensibles
2. **Base de donnees :** Migration vers PostgreSQL ou MySQL pour un environnement de production (modification de `application.properties` et de `pom.xml`)
3. **Pagination et tri :** Mise en place de la pagination (`Pageable`) sur le endpoint `GET /api/patients`
4. **Tests unitaires :** Ajout de tests unitaires avec Mockito (mock du repository) pour tester le service en isolation
5. **Logging :** Integration de SLF4J / Logback avec des niveaux de log configures par environnement

6. **Containerisation :** Dockerisation du service avec un `Dockerfile` et un `docker-compose.yml`
7. **CI/CD :** Pipeline GitHub Actions pour execution automatique des tests et build du JAR a chaque push
8. **Gestion des roles :** Distinguer les acces selon le profil (medecin, administrateur) — seul le medecin verrait le diagnostic
9. **Audit :** Tracer les operations CRUD dans une table d'historique (`@EntityListeners`)